

O Modelo Cliente-Servidor

Programação para a Web 2

Aula 02

Evandro J.R. Silva

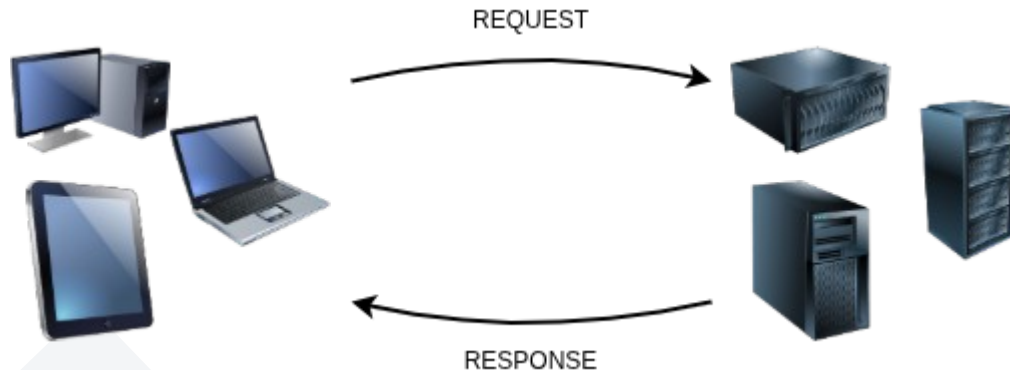
Arquitetura clássica

Cliente

- Máquinas comuns (baixas capacidades de memória, de processamento, etc.)
- Fazem requisições de **recursos**, geralmente através de navegadores web.

Servidor

- Máquinas poderosas (grandes capacidades de memória, de processamento, etc.)
- Recebem as requisições e **respondem** aos *clientes*.



Arquitetura clássica

- Outros componentes
 - A **rede**: permite a comunicação entre cliente e servidor.
 - Os **protocolos**: permitem que cliente e servidor se entendam ao seguirem um conjunto de regras. Os mais comuns:
 - **HTTP/HTTPS** para navegação na web.
 - **SMTP** para troca de e-mails.
 - **FTP** para a transferência de dados (será descontinuado, pois o HTTP já o substituiu na prática).

Arquitetura clássica

- Princípios / Melhores práticas
 - **Modularidade**: o sistema deve ser construído por partes menores (módulos) e independentes. Cada módulo lida com uma tarefa específica.
 - **Escalabilidade**: capacidade de aumento da carga de trabalho.
 - **Escala horizontal**: acréscimo de mais servidores para distribuir a carga.
 - **Escala vertical**: atualização do poder de processamento (CPU + RAM).
 - **Confiabilidade**: a probabilidade do sistema executar suas funções corretamente sem erros em um período de tempo.
 - **Disponibilidade**: a porcentagem de tempo em que o sistema está operacional e acessível aos usuários (objetivo: 99,999% do tempo por ano).
 - **Segurança**: proteção dos dados tanto enquanto estão no servidor (*at rest*) quanto quando estão “em trânsito”.
 - **Otimização de desempenho**: objetivo de minimizar a latência e maximizar o *throughput*.
 - **Manutenibilidade**: o quão fácil o sistema pode ser reparado ou melhorado.

Arquitetura clássica

- Tipos de arquitetura
 - Arquitetura de 1 camada (*1-tier*)
 - Variação onde todas as camadas (*apresentação/interface de usuário*, *lógica de negócios* e o *banco de dados*) residem em um único dispositivo.
 - Arquitetura de 2 camadas (*2-tier*) – a clássica
 - Existem duas partes separadas: *cliente* e *servidor*. O *cliente* lida com a *apresentação* e parte da *lógica de negócios*. O *servidor* fica responsável pelo armazenamento e gerenciamento de dados (ou seja, a maior parte da *lógica de negócios*).
 - Arquitetura de 3 camadas (*3-tier*)
 - Padrão moderno para as aplicações web. O *cliente* fica com a camada de *apresentação*, o processamento da *lógica de negócios* ocorre no *servidor* (*middleware* ou *application server*), e os dados ficam em um *database server*.
 - Arquitetura de múltiplas camadas (*n-tier*)
 - Refere-se a quaisquer variações com mais de 3 camadas.

Tipos de servidores

- Web servers
 - Um computador atendendo a requisições HTTP.
 - Tipicamente esse tipo de servidor se refere a computadores executando um software de servidor web, como Apache, Microsoft IIS (*Internet Information Systems*) ou Nginx.
- Application servers
 - Um computador que hospeda e executa aplicações web, os quais podem ser criados por qualquer tecnologia de desenvolvimento web.

Tipos de servidores

- Database servers
 - Um computador dedicado a executar um *Database Management System* (DBMS), como MySQL, Oracle, MongoDB, etc.
- Mail servers
 - Um computador que cria e responde a requisições de e-mail.
 - Tipicamente executam o protocolo *Simple Mail Transfer Protocol* (SMTP).

Tipos de servidores

- **Media servers**

- Também chamado de *streaming server*, é um servidor dedicado a atender requisições para imagens e vídeos.
- Pode executar *software* especial que permita o *streaming* de conteúdos de vídeo aos clientes.

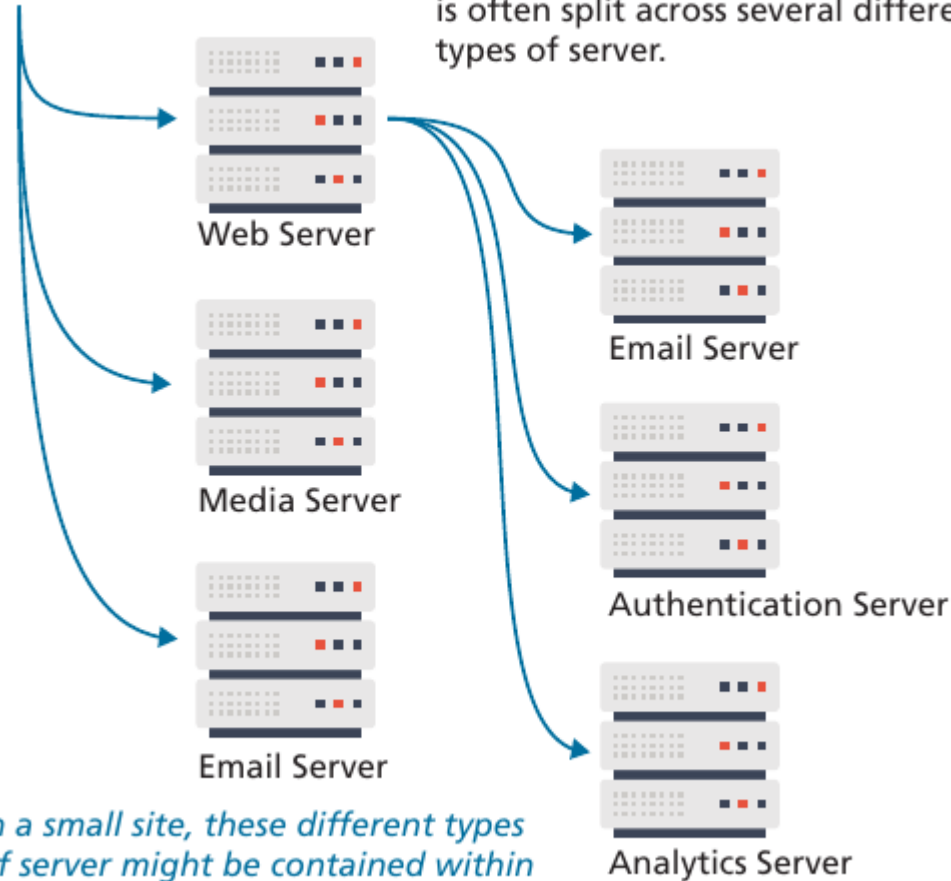
- **Authentication servers**

- Lida com uma das necessidades de segurança mais necessárias de aplicações web.
- Sua execução pode envolver a interação com recursos de rede local, como o *Lightweight Directory Access Protocol* (LDAP) ou *Active Directory*.

Tipos de servidores

Requests

The functionality of a given website is often split across several different types of server.

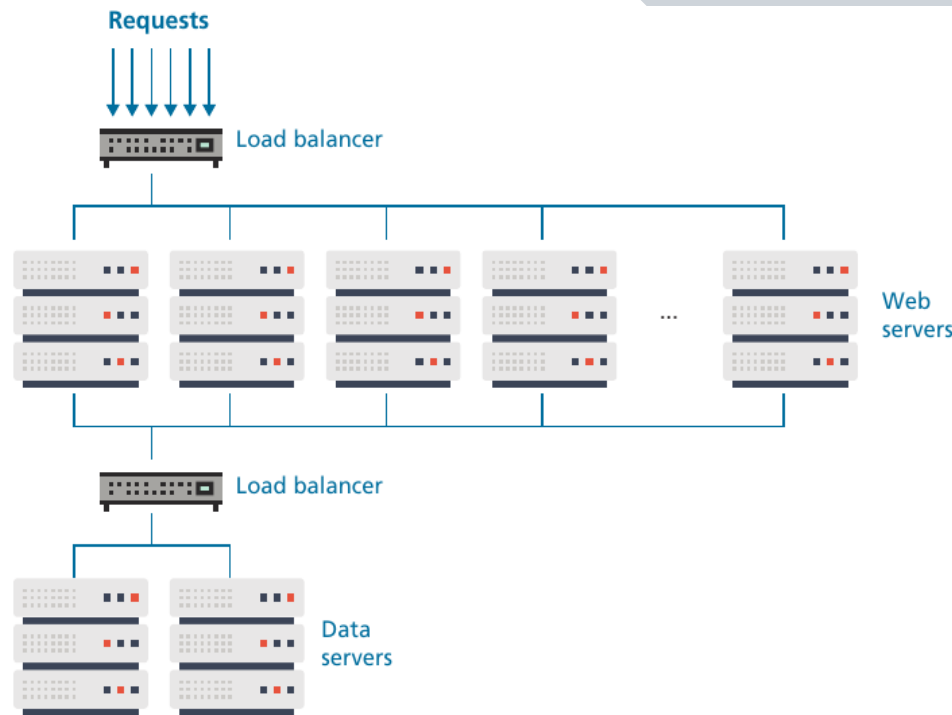


In a small site, these different types of server might be contained within a single machine.

Instalações de servidores

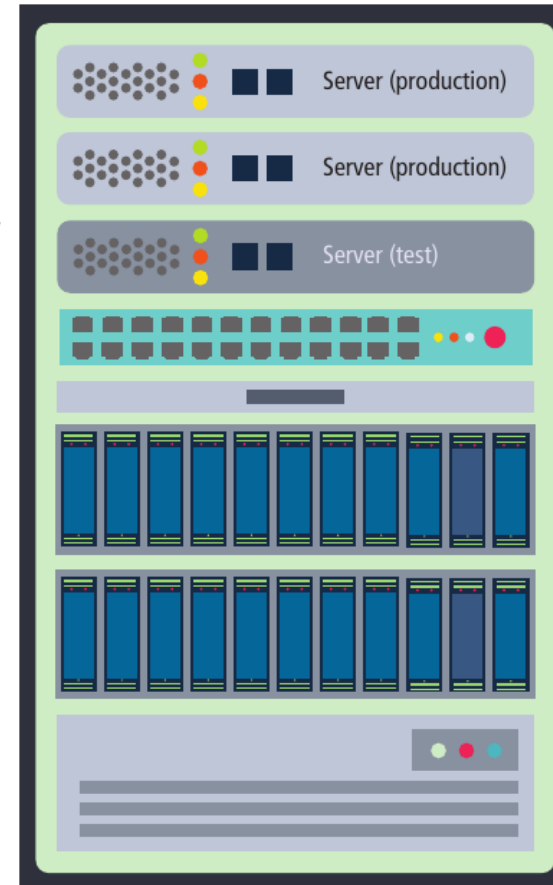
- Server farm

- Um local com um conjunto (*cluster*) de servidores para absorver a grande quantidade de requisições.
- Mesmo quando um único servidor é suficiente para atender à demanda, um *cluster* costuma ser usado para prover *failover redundancy*.



Instalações de servidores

- Server farm
 - Os servidores são organizados em *server racks*.
 - Um *server rack* consiste em um conjunto de hardwares empilhados.



Router and patch panel

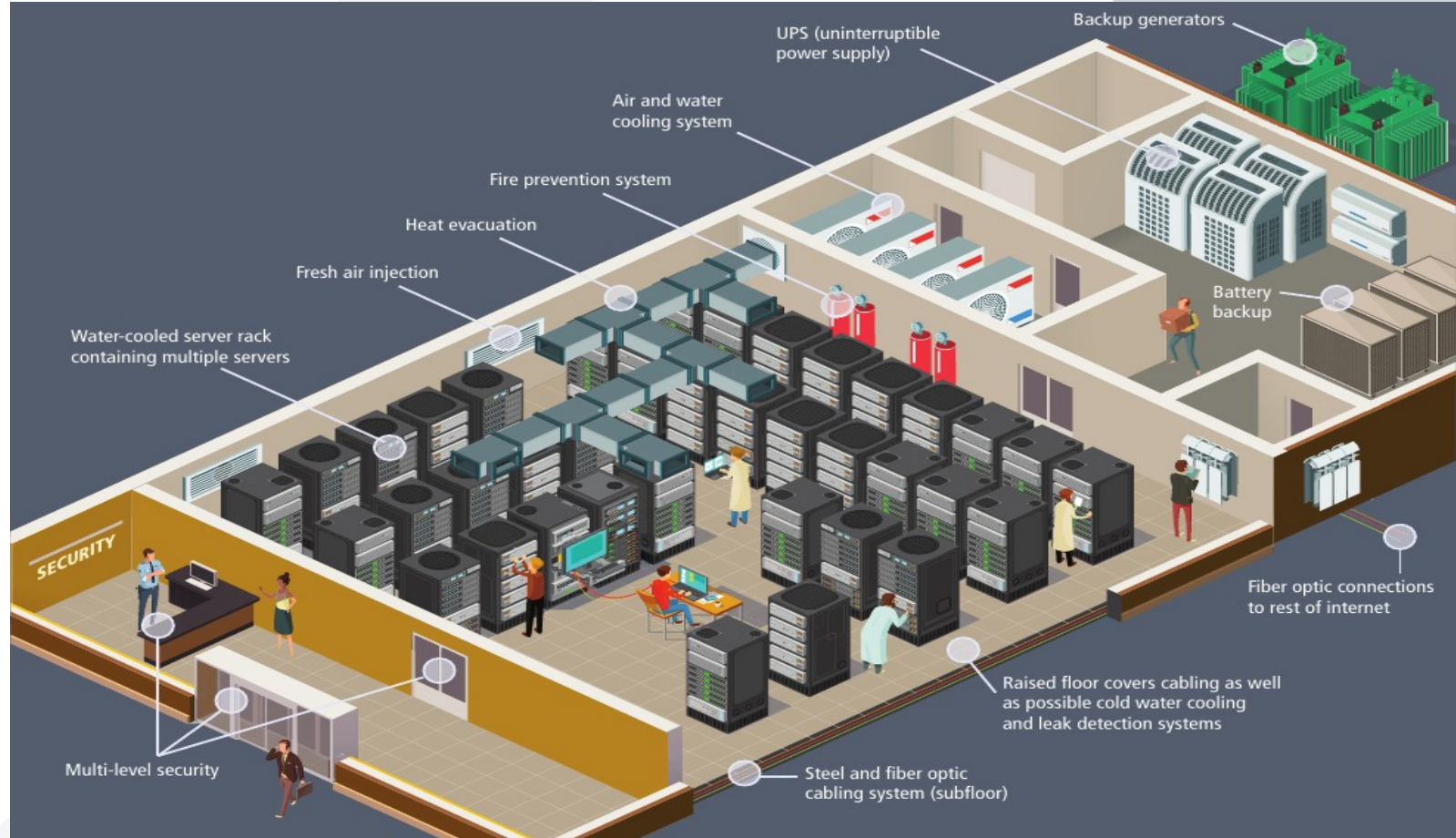
Slide out keyboard and monitor

Hard Drive arrays

UPS

Instalações de servidores

- Data center



Aplicações *stateful* vs. *stateless*

- Uma aplicação *stateful* (com estado) mantém um estado interno ao registrar informações sobre sessões, transações ou qualquer outro tipo de interação.
 - O estado é armazenado em memória ou em um banco de dados, permitindo à aplicação lembrar e rastrear informações específicas de cada usuário ou sessão.

Aplicações *stateful* vs. *stateless*

- Casos de uso de aplicações *stateful*:
 - Esse tipo de aplicação é necessária quando um sistema necessita de contexto persistente, ciclos (*loops*) de feedback contínuos, ou monitoramento contínuo do comportamento do usuário.
 - **Aplicações centradas em usuário** (*user-centric*): aplicações de mídias sociais e *e-commerce*.
 - **Sistemas IoT**: esses sistemas funcionam enviando, recebendo e analisando, continuamente, dados em um *feedback loop*. Baseado em dados históricos ou em tempo real, um dispositivo vai responder à mudança de estado. Por exemplo, um termostato regulando a temperatura de um cômodo.

Aplicações *stateful* vs. *stateless*

- Uma aplicação *stateless* (sem estado) não armazena nenhum estado relacionado com as interações.
 - Cada requisição é tratada de forma independente. O estado é armazenado no lado do cliente ou é transferido em cada solicitação.

Aplicações *stateful* vs. *stateless*

- Casos de uso de aplicações *stateless*:
 - REST APIs.
 - **Microserviços**: permitem que cada uma das principais funcionalidades de uma aplicação maior existam independentemente.
 - **Arquiteturas *serverless***: são planejadas para responder a eventos de forma isolada e não retêm contexto de ações prévias. São ideais para aplicações assíncronas e que podem ser iniciadas instantaneamente.

Trabalho da semana

1. Cada dupla deve elaborar um breve estudo comparativo entre a arquitetura clássica Cliente/Servidor e as arquiteturas modernas (*n-tier*, pelo menos duas), destacando vantagens, limitações e exemplos de uso. E também descrever e comparar pelo menos uma aplicação *stateful* real com uma aplicação *stateless*, também real.
 1. O estudo deve ter pelo menos 3 páginas e incluir as referências utilizadas.
 2. Caso haja uso de IA, deve haver uma seção com a declaração de seu uso, especificando qual(is) e onde foi(ram) utilizada(s), e a finalidade de seu uso.
 3. A entrega consistirá no envio de um arquivo PDF pelo SIGAA (aguardem que eu criarei a tarefa).
 4. O prazo de entrega é a próxima quinta (20/08/26).
 5. Sugestão: usar um template de artigo ou *report* em LaTeX, pelo site *overleaf*.